

Java JDBC Application Development Guide

A Senior Developer's Comprehensive Walkthrough for Linux Environments

Welcome to Java database programming! As a senior Java professional, this guide provides everything you need to build your very first JDBC application from scratch in a **Linux environment**. We will use standard command-line tools (`javac` , `java`), MySQL as our relational database, and the MySQL JDBC Connector/J driver.

Step 1: Install Prerequisites in Linux

Open your terminal and install Java and MySQL (assuming an Ubuntu/Debian-based system such as Ubuntu or Linux Mint). If you are using Fedora or RHEL, substitute `apt` with `dnf` .

1. Install Java Development Kit (JDK)

```
sudo apt update
sudo apt install default-jdk -y
```

Verify that the installation was successful:

```
java -version
javac -version
```

2. Install MySQL Server

```
sudo apt install mysql-server -y
sudo systemctl start mysql
sudo systemctl enable mysql
```

3. Download the MySQL JDBC Driver (Connector/J)

Java requires a specific database driver to communicate with MySQL. Download and extract it via terminal:

```
wget https://dev.mysql.com/get/Downloads/Connector-J/mysql-connector-j-8.3.0.tar.gz
tar -xvzf mysql-connector-j-8.3.0.tar.gz
```

This creates a directory containing `mysql-connector-j-8.3.0.jar` . Keep this file accessible for compilation and execution steps.

Step 2: Set Up the Database and Table

Log into your MySQL console with administrative privileges:

```
sudo mysql -u root
```

Execute the following SQL commands to initialize your schema and create the `student` table:

```
-- Create database
CREATE DATABASE school_db;

-- Use the database
USE school_db;

-- Create student table
CREATE TABLE student (
    rollnumber INT PRIMARY KEY,
    name VARCHAR(50),
    address VARCHAR(100)
);

-- Exit MySQL
EXIT;
```

Step 3: Write the JDBC Java Application

Create a file named `StudentManagement.java` using your preferred text editor (e.g., `nano` or `vim`):

```
nano StudentManagement.java
```

Insert the fully commented, interactive JDBC application code:

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Scanner;

public class StudentManagement {
    static final String DB_URL = "jdbc:mysql://localhost:3306/school_db";
    static final String USER = "root";
    static final String PASS = ""; // Put your MySQL password here if applicable

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
            System.out.println("Connected to the Database successfully!\n");

            int choice;
            do {
                System.out.println("--- Student Management System ---");
                System.out.println("1. Insert Record");
                System.out.println("2. Display Records");
                System.out.println("3. Update Record");
                System.out.println("4. Delete Record");
                System.out.println("5. Exit");
                System.out.print("Enter your choice: ");
                choice = scanner.nextInt();

                switch (choice) {
                    case 1:
                        System.out.print("Enter Roll Number: ");
                        int roll = scanner.nextInt();
                        scanner.nextLine();
                        System.out.print("Enter Name: ");
                        String name = scanner.nextLine();
                        System.out.print("Enter Address: ");
                        String address = scanner.nextLine();

                        String insertQuery = "INSERT INTO student (rollnumber, name, address) VALUES (" + roll + ", " + name + ", " + address + ")";
                        PreparedStatement pstmtInsert = conn.prepareStatement(insertQuery);
                        pstmtInsert.setInt(1, roll);
                        pstmtInsert.setString(2, name);
                        pstmtInsert.setString(3, address);

                        if (pstmtInsert.executeUpdate() > 0) {
                            System.out.println("Student record inserted successfully!\n");
                        }
                        pstmtInsert.close();
                        break;
                }
            } while (choice != 5);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

```

case 2:
    String selectQuery = "SELECT * FROM student";
    Statement stmt = conn.createStatement();
    ResultSet rs = stmt.executeQuery(selectQuery);

    System.out.println("\n--- Student Records ---");
    System.out.println("Roll No\t | Name\t\t | Address");
    System.out.println("-----");
    while (rs.next()) {
        System.out.println(rs.getInt("rollnumber") + "\t | " + rs.getString("name") + "\t | " + rs.getString("address"));
    }
    System.out.println("-----\n");
    rs.close();
    stmt.close();
    break;

case 3:
    System.out.print("Enter Roll Number to update: ");
    int updateRoll = scanner.nextInt();
    scanner.nextLine();
    System.out.print("New Name: ");
    String newName = scanner.nextLine();
    System.out.print("New Address: ");
    String newAddress = scanner.nextLine();

    String updateQuery = "UPDATE student SET name = ?, address = ? WHERE rollnumber = ?";
    PreparedStatement pstmtUpdate = conn.prepareStatement(updateQuery);
    pstmtUpdate.setString(1, newName);
    pstmtUpdate.setString(2, newAddress);
    pstmtUpdate.setInt(3, updateRoll);

    if (pstmtUpdate.executeUpdate() > 0) {
        System.out.println("Student record updated successfully!\n");
    } else {
        System.out.println("Student not found.\n");
    }
    pstmtUpdate.close();
    break;

case 4:
    System.out.print("Enter Roll Number to delete: ");
    int deleteRoll = scanner.nextInt();

    String deleteQuery = "DELETE FROM student WHERE rollnumber = ?";
    PreparedStatement pstmtDelete = conn.prepareStatement(deleteQuery);
    pstmtDelete.setInt(1, deleteRoll);

    if (pstmtDelete.executeUpdate() > 0) {
        System.out.println("Student record deleted successfully!\n");
    } else {
        System.out.println("Student not found.\n");
    }
    pstmtDelete.close();
    break;

```

```

        case 5:
            System.out.println("Exiting application. Thank you!");
            break;

        default:
            System.out.println("Invalid choice! Choose between 1 to 5.\n");
    }
} while (choice != 5);

conn.close();
scanner.close();

} catch (ClassNotFoundException e) {
    System.out.println("MySQL JDBC Driver not found!");
    e.printStackTrace();
} catch (SQLException e) {
    System.out.println("Database connection error or SQL query failure!");
    e.printStackTrace();
}
}
}

```

Step 4: Compile and Execute in the Linux Terminal

To compile a Java file that references external libraries, provide the library path via the Classpath (`-cp`) argument.

1. Compile the Program

```
javac -cp .:mysql-connector-j-8.3.0/mysql-connector-j-8.3.0.jar StudentManagement.java
```

Note for Linux Users: The colon (`:`) is used to separate multiple paths in Linux classpaths (unlike semicolons used on Windows).

2. Run the Program

```
java -cp .:mysql-connector-j-8.3.0/mysql-connector-j-8.3.0.jar StudentManagement
```

Pro-Tips for Linux Developers

- **Class Path Error (`ClassNotFoundException`):** If Java throws this exception, it cannot locate the driver JAR. Verify your directory path to `mysql-connector-j-8.3.0.jar` matches the classpath parameter.
- **Connection Refused:** If connection fails, confirm that your MySQL service is active by running `sudo systemctl status mysql`.